

Kubernetes

Docker gets one container running on one machine. Kubernetes runs many across many machines and keeps them running. This part is the system itself: what the pieces are, and which does what.

1. What Kubernetes is

Kubernetes is a platform that automates the deployment, scaling, and management of containerized applications across multiple machines.

The word doing the work is **automates**. We do not tell Kubernetes to start a container; we tell it what should be true, and it works to make reality match.

```
Desired state → "three copies of payroll:1.4 should be running"
Actual state  → what is running right now
Kubernetes    → closes the gap, forever
```

A node dies and two copies are left. Nobody is paged, because a controller notices and starts a third elsewhere. That loop runs for every object in the cluster, all the time.

It is written K8s, with eight letters replaced by the number. Two things it is not: a container runtime, since it neither builds nor runs containers itself; and a platform anyone can use without learning it, which is why managed layers exist on top.

2. Why Kubernetes

We use Kubernetes to automatically deploy, scale, manage, and recover containerized applications across multiple machines.

Everything below is a problem that appears the moment one machine is not enough.

Problem	What Kubernetes does
A machine dies	Reschedules its containers onto healthy machines
A process crashes	Restarts it, and replaces it if the restarts fail
Traffic doubles	Adds copies, and removes them afterwards
A release goes out, or goes wrong	Rolls pods over gradually, and rolls back on demand
Containers must find each other	Gives each service a stable name and load balancing
Machines have different free space	Packs containers onto nodes by what they request
Configuration differs per environment	Injects it at start, from ConfigMaps and Secrets

The honest summary is that Kubernetes automates operations. Every row above has a manual answer. Kubernetes turns it into something that happens at three in the morning without anybody awake.

It is not always worth it. One service on one machine is better served by Compose. The cost of Kubernetes is real: a control plane to run or pay for, YAML to maintain, and a large surface to learn. It starts paying when there are several services, more than one machine, and an expectation that things stay up without somebody watching.

3. Kubernetes against Docker

Docker is used to build and run containers, while Kubernetes is used to orchestrate and manage containerized applications across a cluster.

Imagine we have a Java application. Docker takes it this far:

```
Java Application
  ↓
Docker Image
  ↓
Docker Container
  ↓
Application runs
```

Docker answers: "How do we package and run our application in a container?"

Kubernetes starts where that leaves off:

```
      Kubernetes
     /   |   \
  Node Node Node
   ↓   ↓   ↓
Container Container Container
```

Kubernetes answers: "How do we manage hundreds of containers across multiple machines?"

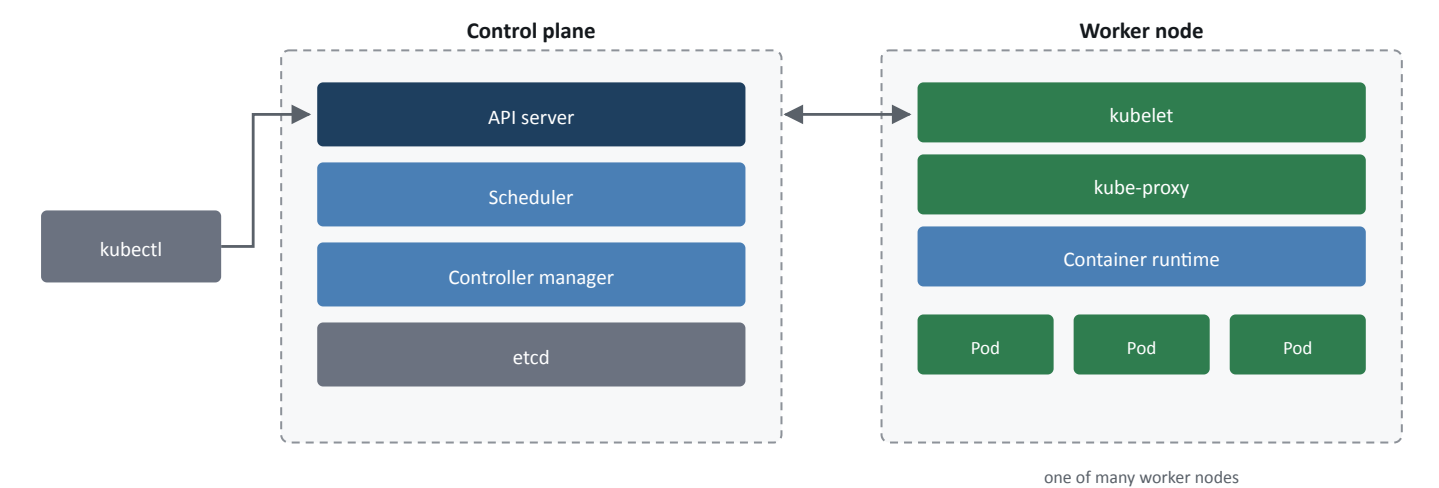
They are not competitors, and the comparison is usually made between the wrong two things.

Compared fairly	Docker	Kubernetes
Builds and runs containers	Both	Runs them through a runtime, builds nothing
Scope	One machine	A group of machines
Restart	A policy on one container	Restart, replace, reschedule
Scaling	More containers on this host	More pods on any host with room
Multi-container app	Compose, on one host	Manifests, across the cluster

Docker produces the image. Kubernetes runs it. They meet at the registry, and the image format is the contract between them, so nothing about a Dockerfile changes when a service moves to Kubernetes.

4. Kubernetes architecture

A cluster has two kinds of machine. The control plane decides what should happen; the worker nodes make it happen.



Every arrow ends at the API server. No component talks to any other component directly, and only the API server reads or writes etcd.

The API server is the only front door. `kubectl` goes through it, the kubelet goes through it, and every controller goes through it. That single rule explains most of the design: components do not know about each other, so any of them can restart without the others noticing.

5. Control plane against worker node

Control plane	Worker node
Decides what should run and where	Runs it
API server, scheduler, controllers, etcd	kubelet, kube-proxy, container runtime
Holds all cluster state	Holds no state worth keeping
Usually 3 machines, for quorum	As many as the workload needs
Managed for us on EKS, GKE and AKS	Ours to size and pay for

Losing the control plane does not stop the workload. Pods already running keep running and keep serving traffic. What stops is change: no scheduling, no rollouts, no self-healing, and `kubectl` returns nothing.

Control plane machines usually run no application workload. On a managed cluster we cannot put anything there at all. On a self-managed one a taint keeps application pods off, so a busy service cannot starve the scheduler.

6. The Kubernetes cluster

A Kubernetes cluster is a group of machines that Kubernetes manages together to run containerized applications.

Those machines are the control plane and its worker nodes, and we deploy to the whole group as though it were one machine.

Control plane machines come in odd numbers. etcd elects a leader by majority, so three machines tolerate one failure and five tolerate two. Two machines tolerate none, because one failure leaves no majority, which makes a pair strictly worse than a single machine for availability.

Cluster	What we operate
Managed (EKS, GKE, AKS)	Worker nodes only. The control plane is a paid service
Self-managed	Everything, including etcd backups and version upgrades
Local (kind, minikube)	A whole cluster in containers or a VM, for development

Whichever it is, every machine appears in one listing:

```
kubectl get nodes
```

Output:

NAME	STATUS	ROLES	AGE	VERSION
kind-dev-cp	Ready	control-plane	6d	v1.31.2
kind-dev-w1	Ready	<none>	6d	v1.31.2
kind-dev-w2	Ready	<none>	6d	v1.31.2

The **ROLES** column is the only thing separating the two kinds. A control plane machine is an ordinary node that also runs the control plane components.

Namespaces divide one cluster logically. They scope names, quotas and access, which is how one cluster carries several teams or environments. They are not a security boundary on their own; that takes network policy and RBAC.

```
kubectl get pods -n payroll
kubectl config set-context --current --namespace=payroll
```

7. Kubernetes API Server

The API server is a REST service over HTTPS, and it is the only component that reads or writes etcd.

Every request passes through the same four stages:

```
Authentication → who is this?
Authorization → is this account allowed to do it? (RBAC)
Admission      → should it be changed or rejected first?
Validation     → is the object well formed? → written to etcd
```

Admission control is where a cluster's rules live. It is the stage that injects a sidecar, sets a default memory limit, or rejects an image from an untrusted registry, and it happens before anything is stored.

```
kubectl get --raw /healthz
kubectl auth can-i create deployments -n payroll
```

Nothing bypasses it, which is why every audit log, every permission check and every quota is enforced in one place. It also means the API server is the piece that must scale, and the piece worth watching when a cluster feels slow.

8. Scheduler

Scheduler is a Kubernetes control-plane component that decides which worker node should run a Pod.

It watches for pods that have no node assigned, picks one for each, and writes the choice back.

It runs in two phases:

Phase	What happens
Filtering	Remove every node that cannot take the pod at all
Scoring	Rank what is left, and take the best

Filtering is about hard facts: does the node have enough unreserved CPU and memory for the pod's **requests**, does it tolerate the node's taints, does it match the node selector, are the required ports free.

Scoring spreads the work: prefer nodes with more room, prefer spreading replicas of the same Deployment across nodes and zones, honor affinity rules.

```
spec:
  nodeSelector:
    disktype: ssd
  tolerations:
    - key: "gpu"
      operator: "Exists"
```

Requests are what the scheduler reads, not limits and not actual use. A pod with no requests can be placed on a node that is already full, which is how one service ends up starving another.

The scheduler only writes a node name. It starts nothing. The kubelet on that node notices the pod is now its responsibility and takes it from there.

9. Controller Manager

Controller Manager continuously checks the cluster and takes action to make the actual state match the desired state.

It does that as a set of control loops. Each controller watches one kind of object and works to close the gap between spec and status.

```
observe → compare desired with actual → act → observe again
```

Controller	What it reconciles
Deployment	Creates and updates ReplicaSets for a rollout
ReplicaSet	Keeps the right number of pods alive
Node	Notifies unreachable nodes and evicts their pods
Job	Runs pods to completion, and retries failures
Endpoints	Keeps a Service's list of pod addresses current

They all ship in one binary, `kube-controller-manager`, but behave as separate loops.

This loop is the whole model, not an implementation detail. Deleting a pod by hand from a Deployment does not remove it; the ReplicaSet controller sees one fewer than it wants and creates another within seconds. Changing the Deployment is the only thing that changes the outcome.

The **cloud controller manager** is the same idea for the parts that are provider-specific: creating a load balancer for a `LoadBalancer` Service, attaching a disk, removing a node that the provider deleted.

10. etcd

etcd is a distributed key-value store that Kubernetes uses to store the cluster's state and configuration.

That means every object, every spec and every status.

It is the only stateful component. Everything else can be restarted, and rebuilds what it needs by reading from the API server.

Property	Detail
Consistency	Raft consensus. Writes need a majority
Members	An odd number, usually three or five
Access	Only the API server. Nothing else connects to it
Contents	Objects, and Secrets, which are not encrypted by default

```
ETCDCTL_API=3 etcdctl snapshot save /backup/etcd-$(date +%F).db
```

Losing etcd is losing the cluster, not the workload. Running pods keep running, and nothing can be changed, recovered or rebuilt until a snapshot is restored. On a self-managed cluster a tested etcd backup is the single most important piece of operations work; on a managed cluster the provider does it.

Turn on encryption at rest. Secrets are stored in etcd, and by default they are only base64-encoded there.

11. Kubelet

The kubelet is the agent on every node, control plane machines included.

What it does, in order:

```
watch the API server for pods assigned to this node
  → tell the container runtime to pull and start them
  → run their probes
  → report status back to the API server
```

Responsibility	Detail
Starting containers	Through the CRI, never by itself
Probes	Readiness, liveness and startup, on a timer
Restarts	Applies the pod's <code>restartPolicy</code> , with a backoff
Mounting	Attaches volumes before the container starts
Reporting	Node conditions, capacity, and pod status

The kubelet only manages pods the API server assigned to it. A container started on the node with `docker run` is invisible to Kubernetes, and a pod the kubelet does not recognize is cleaned up.

A node whose kubelet stops reporting is marked `NotReady`. After a grace period the node controller evicts its pods so they can be rescheduled, which is what makes a dead machine recover on its own.

12. Kube-proxy

kube-proxy makes Services work on each node. When a pod connects to `payroll:8080`, kube-proxy is the reason that address reaches a real pod.

It watches Services and their endpoints, and programs the node's network rules to match.

Mode	How
iptables	Rules that rewrite the destination to a random matching pod. The default
IPVS	A kernel load balancer. Better with thousands of Services
eBPF	Provided by a CNI such as Cilium, replacing kube-proxy entirely

In iptables mode, no traffic passes through kube-proxy. It writes rules and steps aside; the kernel does the work. This surprises people who expect a proxy to be in the path, and it is why kube-proxy restarting does not drop connections.

Load balancing is random, not round robin, and there is no awareness of how busy a pod is. Anything smarter than that is what a service mesh is for.

13. Container runtime

Kubernetes runs no containers itself. The kubelet asks a runtime through the Container Runtime Interface, a small API that any runtime can implement. That is what makes the runtime a replaceable part rather than a fixed dependency.

```
kubelet → CRI → containerd → runc → the container
```

Runtime	Note
containerd	The usual default, and what Docker uses underneath
CRI-O	Built only for Kubernetes
Docker Engine	Removed as a direct option in Kubernetes 1.24

"Kubernetes deprecated Docker" is about the adapter, not the images. What went was `dockershim`, the shim that let the kubelet drive Docker Engine. Kubernetes talks to `containerd` directly now, and images built with Docker still run everywhere because the image format is an OCI standard.

14. Kubernetes objects and resources

An object is a record of intent stored in etcd. Every one of them has the same four top-level fields.

```
apiVersion: apps/v1      # which API group and version
kind: Deployment          # what sort of object
metadata:                 # name, namespace, labels, annotations
  name: payroll
spec:                     # what we want
  replicas: 3
status:                   # what is actually true, written by Kubernetes
```

`spec` is ours and `status` is the cluster's. We never write `status`, and the difference between the two is exactly what the controllers are working to remove.

Category	Objects
Workloads	Pod, Deployment, ReplicaSet, StatefulSet, DaemonSet, Job, CronJob
Networking	Service, Ingress, NetworkPolicy
Configuration	ConfigMap, Secret
Storage	PersistentVolume, PersistentVolumeClaim, StorageClass
Cluster	Namespace, Node, ServiceAccount, Role, RoleBinding

```
kubectl api-resources
kubectl explain deployment.spec.strategy
```

`kubectl explain` is the documentation for every field, offline and matched to the cluster's own version.

A resource is the API endpoint; an object is one instance of it. The words are used for each other constantly, and the distinction almost never matters outside a conversation about the API itself.

15. What a Pod is

A Pod is the smallest thing Kubernetes creates and runs. It holds one or more containers that are always placed on the same node and share an address.

```
apiVersion: v1
kind: Pod
metadata:
  name: payroll
spec:
  containers:
  - name: payroll
    image: ghcr.io/acme/payroll:1.4
    ports:
    - containerPort: 8080
```

What the containers in a Pod share, and what stays their own:

Shared by every container in a Pod	Kept separate
One IP address, and the port space	The filesystem, from each container's own image
<code>localhost</code> between them	Process names and ids
Volumes, when mounted into both	Resource requests and limits
The lifecycle: started together, stopped together	—

A Pod gets one IP address, not one per container. Two containers in a Pod reach each other on `localhost`, and they cannot both listen on port 8080.

Nothing outside should ever hold a Pod's address. Pods are replaced constantly and the new one has a different IP, which is what Services exist to hide.

16. Why Kubernetes uses Pods

The obvious design would schedule containers directly. Kubernetes adds a wrapper around them, and it buys four things.

The Pod gives	Which matters because
A unit of scheduling	Containers that must sit together always do
A unit of addressing	One IP for the group, so helpers share it
A unit of lifecycle	They start, stop and are replaced as one
A place for shared storage	A volume can be written by one and read by another

The case it was designed for is a helper container beside the application: a proxy, a log shipper, a credential refresher. Those need the application's network and its files, and they are useless on their own.

Scheduling them separately would not work. Two containers on different nodes cannot share `localhost` or a volume, and nothing would keep them together. Making the Pod the atom removes the problem instead of solving it repeatedly.

It also keeps the runtime simple. The runtime is asked to start containers; everything about how they relate lives in Kubernetes. That separation is why the runtime can be swapped without changing anything above it.

17. Pod against container

Pod	Container
A Kubernetes object, stored in etcd	A process the runtime creates
Has an IP, a name and a namespace	Has neither; it uses the Pod's
Scheduled onto a node	Started where its Pod already is
Holds one or more containers	Holds one image
Created by a Deployment, Job or us	Created by the kubelet, never directly

There is no container object in Kubernetes. `kubectl` cannot create, delete or list containers on their own, because the API has no such resource. The Pod is the smallest thing the API knows about, and every container command is really a Pod command with `-c` naming one of them.

```
kubectl logs payroll -c log-shipper
kubectl exec -it payroll -c payroll -- sh
```

18. Single-container against multi-container Pods

One container per Pod is the normal case, and it is the right answer unless a second container genuinely needs the first one's network or files.

Pattern	The second container
Sidecar	Runs alongside: ships logs, refreshes certificates, serves metrics
Ambassador	Proxies the application's outbound connections
Adapter	Reshapes the application's output into a standard format
Init container	Runs to completion first, then exits

```
spec:
  initContainers:
    - name: wait-for-db
      image: busybox:1.36
      command: ['sh', '-c', 'until nc -z db 5432; do sleep 2; done']
```

Then the containers that stay running, in the same Pod spec:

```
containers:
  - name: payroll
    image: ghcr.io/acme/payroll:1.4
  - name: log-shipper
    image: fluent-bit:3.1
    volumeMounts:
      - name: logs
        mountPath: /var/log/app
```

Init containers run in order and must each succeed. The application containers do not start until every one has exited cleanly, which is how a migration or a dependency wait is expressed without putting it in the application.

Two services do not belong in one Pod. They would scale together, be released together and fail together. The test is whether the second container is useful without the first; if it is, it wants its own Deployment.

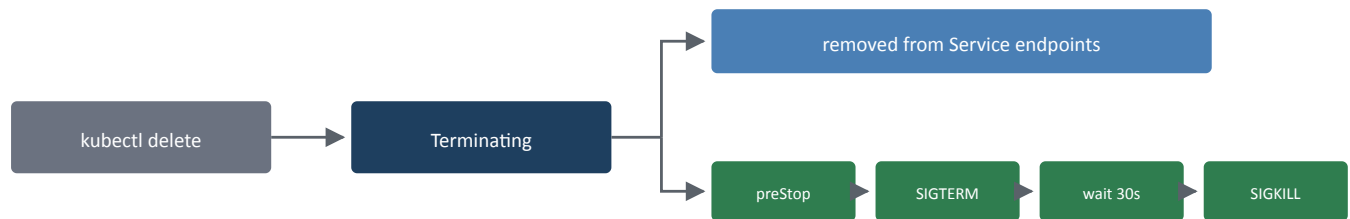
19. The Pod lifecycle

Starting a Pod is a sequence, and each step can fail visibly.

accepted by the API server

- scheduled onto a node
- images pulled
- init containers, in order
- application containers started
- readiness probe passes → traffic

Stopping one is a sequence too, and this is the half that gets misconfigured.



The two branches run at the same time and neither waits for the other. That race is why a Pod can still receive requests after it has been told to stop.

Endpoint removal is not ordered before `SIGTERM`. Kubernetes tells the kubelet to stop the containers and tells kube-proxy to drop the endpoint at the same moment, and the rules take a moment to propagate. An application that exists the instant it sees `SIGTERM` therefore drops in-flight requests.

The fix is a short `preStop` sleep, which holds the container open while the endpoint removal reaches every node:

```
lifecycle:
  preStop:
    exec:
      command: ["sh", "-c", "sleep 5"]
  terminationGracePeriodSeconds: 45
```

The grace period covers the whole stop, `preStop` included. A 5 second hook inside a 30 second grace period leaves the application 25 seconds, not 30.

20. Pod states

Two different things are called the state of a Pod, and mixing them up is the usual confusion.

The phase is one of five values, and it is deliberately coarse:

Phase	Meaning
Pending	Accepted, but not yet running. Scheduling, or pulling images
Running	Bound to a node, and at least one container is running
Succeeded	Every container exited with 0 and will not restart
Failed	Every container stopped, and at least one failed
Unknown	The node cannot be reached

The **STATUS** column is not the phase. `kubectl get pods` prints something more useful: the reason a container is waiting, or the fact that the Pod is shutting down.

NAME	READY	STATUS	RESTARTS	AGE
payroll-7d4b9c8f5d-x2plq	1/1	Running	0	4h
payroll-7d4b9c8f5d-m9kzt	0/1	CrashLoopBackOff	7	22m
payroll-7d4b9c8f5d-q4tns	0/1	ImagePullBackOff	0	3m
migrate-once-lp82x	0/1	Completed	0	1d

`CrashLoopBackOff` and `ImagePullBackOff` are container reasons inside a Pod whose phase is still `Pending` or `Running`. `Completed` is what `Succeeded` looks like in that column.

Conditions are the third answer, and they are the ones a Service reads. Unlike the phase, they are independent flags, each recording whether one step has finished:

Condition	True when
PodScheduled	A node has been chosen
Initialized	Every init container finished
ContainersReady	Every container is ready
Ready	The Pod should receive traffic

`READY 0/1` means one container exists and none is ready, which is a readiness probe failing rather than a crash.

21. Pod restart behavior

`restartPolicy` is set on the Pod and applies to its containers.

Policy	Restarts a container that	Used by
Always	Exits for any reason, including success	Deployments. The default
OnFailure	Exits with a non-zero code	Jobs and CronJobs
Never	Never	One-off diagnostic Pods

Restarting is not rescheduling. The Pod keeps its name, its node and its IP address; only the process inside starts again. Moving to another node means a new Pod, which only a controller creates.

The delay doubles. Ten seconds, then twenty, forty, and so on to a five minute ceiling, and it resets once a container has run for ten minutes. That backoff is the `BackOff` in `CrashLoopBackOff`.

```
RESTARTS
0          healthy
3 (2m ago) recovering, or failing slowly
47         has been failing for hours
```

A climbing `RESTARTS` count on a `Running` Pod is the quiet failure. The Pod looks healthy in every status check, and it is crashing and recovering the whole time. It is worth alerting on directly.

22. What a Deployment is

A Deployment declares the desired state for a set of identical Pods and keeps it true.

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: payroll
spec:
  replicas: 3
  selector:
    matchLabels:
      app: payroll
```

The rest is the Pod template from section 15, with the labels the selector looks for:

```
template:
  metadata:
    labels:
      app: payroll
  spec:
    containers:
      - name: payroll
        image: ghcr.io/acme/payroll:1.4
```

It does not create Pods directly. It creates a ReplicaSet, and the ReplicaSet creates the Pods, which is what makes a rollout possible: a change produces a second ReplicaSet, and the two are resized in opposite directions.

```
kubectl rollout status deployment/payroll
kubectl rollout history deployment/payroll
kubectl rollout undo deployment/payroll
```

Rollback works because the old ReplicaSet is kept, scaled to zero, with its own pod template. Undoing a rollout scales it back up; nothing is rebuilt and no image is pulled again.

23. Deployment against Pod

Pod	Deployment
One instance	A declared number of them
Gone for good if its node dies	Replaced within seconds
Changing the image means deleting it	Changing the image starts a rollout
No history	Revision history, and <code>rollout undo</code>
No scaling	<code>kubectl scale</code> , or an autoscaler

Never deploy a bare Pod. Nothing watches it, so a node failure or an evicted Pod is simply the end of that workload. Bare Pods are for debugging, and the `--dry-run` flag from section 25 is the right way to start from one.

Deleting a Deployment's Pod proves the point. Within a second or two there is another one with a new name, because the ReplicaSet controller counted and found one missing.

24. Deployment strategies

The strategy decides what happens between the old version and the new one.

```
spec:
  strategy:
    type: RollingUpdate
    rollingUpdate:
      maxSurge: 1
      maxUnavailable: 0
```

Strategy	What it does	Cost
RollingUpdate	Replaces Pods gradually. The default	Both versions run at once
Recreate	Stops all old Pods, then starts new ones	Downtime

maxUnavailable: 0 is the setting for no downtime. With **maxSurge: 1** Kubernetes adds one new Pod, waits for it to become ready, then removes one old Pod, and repeats. Nothing goes away before its replacement is serving.

Recreate exists for a reason. Two versions cannot run at once against a schema that only one of them understands, or against a volume that only one writer can hold. Accepting a few seconds of downtime is the correct answer there.

Both versions running at once is a constraint on the code. During a rolling update, requests are served by old and new Pods together, so a release must be compatible with the one before it. That is the same rule that makes a rollback safe.

Two strategies people expect and Kubernetes does not provide on its own:

Strategy	How it is actually done
Blue-green	Two Deployments, and a Service selector moved between them
Canary	A second Deployment with few replicas, or a mesh splitting traffic by weight

Argo Rollouts and Flagger exist to add these, and a service mesh gives the traffic splitting that weighted canaries need.

25. Declarative against imperative

Two ways to work, and Kubernetes is built for one of them. The difference is not a preference about tooling; it decides whether the cluster can be rebuilt from a repository or only from memory. Both are used in practice, and the line between them is how long the change is meant to last.

	Imperative	Declarative
We say	Do this now	This is how it should be
Command	<code>kubectl create</code> , <code>run</code> , <code>scale</code> , <code>delete</code>	<code>kubectl apply -f</code>
Lives in	Shell history	Files, in git
Run twice	Fails the second time	Same result, no error
Records why	Nothing	The commit and its review

```
kubectl create deployment payroll --image=payroll:1.4
kubectl scale deployment payroll --replicas=5
```

```
kubectl apply -f k8s/
kubectl diff -f k8s/           # what would change, before changing it
```

Declarative wins for anything that lasts. The manifest is reviewed, versioned and reproducible, a rollback is a revert, and applying it again when somebody has changed something by hand puts it back. This is what GitOps tools automate.

apply is idempotent and **create** is not. Running **create** on something that exists is an error; **apply** compares the file with the live object and changes only the difference.

Imperative is still right for short-lived things: getting a debug pod up, scaling during an incident, or looking at the cluster. The rule that keeps both usable is that nothing imperative should survive the day it was typed.

```
kubectl create deployment payroll --image=payroll:1.4 \
--dry-run=client -o yaml > deployment.yaml
```

That generates the manifest rather than the object, which is the fastest honest way to start a declarative file.

The pieces together

Piece	What it answers
API server	The only way in, and the only writer of etcd
etcd	Where all state lives
Scheduler	Which node a pod goes on
Controller manager	Keeping actual equal to desired
Kubelet	Starting and watching containers on one node
kube-proxy	Making a Service address reach a pod
Container runtime	Actually creating the container
Object	A record of intent, with spec and status
Pod	The smallest thing scheduled, and what holds an IP
Phase and STATUS	Coarse state against the reason something is stuck
<code>restartPolicy</code>	Whether a container comes back, and how fast
Deployment	How many Pods, which image, and how to roll back

Piece	What it answers
Strategy	What happens between the old version and the new
Declarative	Why the manifest, not the command, is the source of truth

For example, `kubectl apply -f deployment.yaml`. The API server authenticates the request, checks RBAC, runs admission, validates the object and writes it to etcd. The Deployment controller sees it and creates a ReplicaSet; the ReplicaSet controller sees that and creates three pods with no node. The scheduler filters and scores the nodes and writes a node name onto each pod. Each kubelet sees a pod assigned to it, asks containerd to pull the image and start it, and reports back. kube-proxy programs the rules that make the Service address reach the new pods.

Nine components, one path. Being able to say that sequence out loud is most of what a Kubernetes architecture question is asking for, and every failure mode is easier to place once the order is clear.